

DBMS Basics

These notes cover core DBMS placement topics: DBMS vs file system, schema vs instance, data models, constraints, and ER model.

1. What Is DBMS?

A Database Management System, or DBMS, is software used to store, organize, manage, retrieve, and secure data efficiently.

Examples:

- MySQL
- PostgreSQL
- Oracle Database
- SQL Server
- SQLite
- MongoDB

A database is the organized collection of data. A DBMS is the software that manages that data.

Main Goals Of DBMS

- Store large amounts of data efficiently.
- Retrieve data quickly using queries.
- Avoid unnecessary duplication.
- Maintain data consistency.
- Support multiple users at the same time.
- Provide security and access control.
- Recover data after failures.
- Enforce rules using constraints.
- Maintain relationships between data.

Example

In a college placement system:

- Student details are stored in a **Student** table.
- Company details are stored in a **Company** table.
- Applications are stored in an **Application** table.
- Relationships show which student applied to which company.

The DBMS ensures that invalid data, such as an application for a non-existing student, is not inserted.

2. File System

A file system stores data in separate files managed by the operating system.

Examples:

- Text files
- CSV files
- Excel files
- Binary files

In a file system approach, each application usually manages its own files and logic for reading, writing, searching, updating, and securing data.

Example

A college may store data like this:

- `students.csv`
- `companies.csv`
- `applications.csv`

The program must manually check whether a student exists before adding an application. It must also manually prevent duplicates, handle security, and manage backups.

3. DBMS Vs File System

Feature	File System	DBMS
Data storage	Stored in separate files	Stored in organized databases
Data redundancy	High	Controlled
Data inconsistency	Common	Reduced using constraints and normalization
Data sharing	Difficult	Easy
Data security	Limited	Strong access control
Backup and recovery	Mostly manual	Built-in mechanisms
Concurrent access	Poor support	Strong support using transactions
Querying	Requires custom code	SQL/query language support
Data independence	Low	High
Integrity constraints	Hard to enforce	Built-in constraints
Relationships	Manually maintained	Properly modeled
Atomicity	Difficult	Supported through transactions
Scalability	Limited	Better for large systems

Advantages Of DBMS Over File System

1. Reduced Data Redundancy

Redundancy means storing the same data multiple times.

In a file system, student data may be repeated in many files:

- Fee file
- Library file
- Placement file
- Hostel file

In a DBMS, student details can be stored once in a **Student** table and referenced elsewhere.

2. Improved Data Consistency

If a student's phone number is stored in five files, updating only one file causes inconsistency.

In DBMS, centralized storage reduces this problem.

3. Better Data Sharing

Multiple users and applications can access the same database.

Example:

- Placement officer checks eligible students.
- Faculty updates marks.
- Student views profile.

All can use the same database with different permissions.

4. Better Security

DBMS supports:

- User accounts
- Roles
- Permissions
- Views
- Authentication
- Authorization

Example:

A student may view only their own data, while an admin can update all records.

5. Backup And Recovery

DBMS can recover data after:

- System crash
- Power failure
- Transaction failure
- Disk failure

Recovery is usually based on logs, checkpoints, and backups.

6. Concurrency Control

DBMS allows multiple users to access data simultaneously while preserving correctness.

Example:

Two students applying for the last available interview slot should not both get it if only one slot exists.

7. Data Integrity

DBMS enforces rules using constraints.

Example:

- Age must be positive.
- Email must be unique.
- Department ID must exist before assigning it to a student.

8. Data Independence

DBMS separates data structure from application logic.

If internal storage changes, application programs may not need to change.

Disadvantages Of DBMS

- More complex than simple files.
- Requires installation and administration.
- Can be costly for enterprise systems.
- Needs trained users such as DBAs.
- Failure of DBMS may affect many applications.
- Overhead may be unnecessary for very small data.

When File System Is Enough

A file system may be enough when:

- Data is small.
- Only one user uses it.
- No complex querying is required.
- Security and consistency are not major concerns.
- The data is temporary.

Example:

A simple personal notes app can use files.

When DBMS Is Better

DBMS is better when:

- Data is large.

- Multiple users access it.
- Data relationships matter.
- Security is important.
- Queries are frequent.
- Data consistency is important.
- Backup and recovery are needed.

Example:

Banking, railway reservation, college management, e-commerce, hospital systems.

4. Important DBMS Terminology

Data

Raw facts without context.

Example:

101, Prabhask, CSE, 8.7

Information

Processed data with meaning.

Example:

Student Prabhask from CSE has CGPA 8.7.

Database

An organized collection of related data.

DBMS

Software used to define, create, store, retrieve, update, and manage databases.

Metadata

Data about data.

Example:

For a Student table:

- Column names
- Data types
- Constraints
- Table structure

Data Dictionary

A repository that stores metadata about the database.

It may contain:

- Table names
- Attribute names
- Data types
- Constraints
- Relationships
- User permissions

Table / Relation

A table is a collection of rows and columns.

In the relational model, a table is called a relation.

Row / Tuple

A single record in a table.

Example:

One student record.

Column / Attribute

A property or field of an entity.

Example:

`student_id, name, cgpa`

Domain

The set of valid values for an attribute.

Example:

The domain of `gender` may be `{Male, Female, Other}`.

The domain of `cgpa` may be `0.00` to `10.00`.

5. Schema Vs Instance

Schema

Schema is the logical structure or design of a database.

It defines:

- Tables
- Attributes
- Data types
- Constraints

- Relationships
- Views

Schema is also called the database blueprint.

Example Schema

```
Student(student_id, name, department, cgpa)
Company(company_id, company_name, package)
Application(application_id, student_id, company_id, status)
```

This only describes the structure, not the actual data.

Instance

An instance is the actual data stored in the database at a particular moment.

Example Instance

For **Student**:

student_id	name	department	cgpa
1	Aman	CSE	8.5
2	Riya	ECE	8.9
3	Kabir	IT	7.8

This data can change frequently.

Schema Vs Instance Table

Basis	Schema	Instance
Meaning	Structure of database	Data stored at a specific time
Changes	Rarely changes	Changes frequently
Also called	Intension	Extension
Example	Table design	Rows inside tables
Defined by	DDL	DML
Time dependency	Time independent	Time dependent

Easy Analogy

Schema is like the format of a blank form.

Instance is like the filled form at a particular time.

Types Of Schema

1. Physical Schema

Describes how data is physically stored.

Example:

- Indexes
- File organization
- Storage blocks

2. Logical Schema

Describes the logical design of the database.

Example:

- Tables
- Columns
- Relationships
- Constraints

3. View Schema / External Schema

Describes how different users see the database.

Example:

A student sees only their marks, while an admin sees all student records.

6. Three-Schema Architecture

DBMS commonly uses three levels of abstraction.

1. Internal Level

Lowest level.

Describes physical storage:

- How data is stored on disk
- Indexing
- File organization
- Compression
- Access paths

2. Conceptual Level

Middle level.

Describes the complete logical structure of the database:

- Entities
- Attributes
- Relationships
- Constraints

This is what database designers usually work with.

3. External Level

Highest level.

Describes user-specific views.

Different users can see different parts of the same database.

Why Three-Schema Architecture Is Useful

- Provides data abstraction.
- Improves security.
- Supports multiple user views.
- Provides data independence.

7. Data Independence

Data independence means the ability to change schema at one level without changing the schema at the next higher level.

1. Physical Data Independence

Ability to change physical storage without changing the logical schema.

Example:

- Changing file organization
- Adding indexes
- Changing storage device
- Changing compression method

Applications should continue to work.

2. Logical Data Independence

Ability to change logical schema without changing external views or applications.

Example:

- Adding a new column
- Splitting a table
- Merging tables

Logical data independence is harder to achieve than physical data independence.

8. Data Models

A data model defines how data is organized, stored, related, and manipulated in a database.

It provides a way to describe:

- Data structure
- Relationships
- Constraints
- Operations

Main Types Of Data Models

1. Hierarchical data model
2. Network data model
3. Relational data model
4. Entity-Relationship model
5. Object-oriented data model
6. Object-relational data model
7. Semi-structured data model
8. NoSQL data models

9. Hierarchical Data Model

The hierarchical model organizes data in a tree-like structure.

Each child has only one parent.

Example

```
College
  Department
    Student
```

Features

- Parent-child relationship.
- One-to-many relationships are natural.
- Data is accessed from root to leaf.
- Uses pointers or links internally.

Advantages

- Simple structure.
- Fast for one-to-many hierarchical data.
- Good for fixed relationships.

Disadvantages

- Many-to-many relationships are difficult.
- Data redundancy can occur.
- Structure is rigid.
- Difficult to reorganize.

Real Example

IBM IMS used hierarchical model.

10. Network Data Model

The network model organizes data as a graph.

A child can have multiple parents.

Example

A student can enroll in many courses, and a course can have many students.

Student <--> Course

Features

- Supports many-to-many relationships.
- Uses records and links.
- More flexible than hierarchical model.

Advantages

- Handles complex relationships.
- Faster access using links.

Disadvantages

- Complex design.
- Difficult to maintain.
- Structural changes are hard.

11. Relational Data Model

The relational model represents data as tables, also called relations.

Introduced by E. F. Codd.

Example

Student

student_id	name	department	cgpa
1	Aman	CSE	8.5
2	Riya	ECE	8.9

Key Concepts

- Relation: table
- Tuple: row
- Attribute: column
- Domain: set of allowed values
- Degree: number of attributes
- Cardinality: number of tuples
- Key: attribute or set of attributes used to identify tuples

Advantages

- Simple table-based structure.
- SQL support.
- Strong mathematical foundation.
- Easy to understand.
- Supports data independence.
- Reduces redundancy using normalization.

Disadvantages

- Joins can be expensive.
- Not ideal for highly nested or graph-like data.
- Object-relational mismatch in applications.

12. Entity-Relationship Data Model

The ER model is a high-level conceptual model used during database design.

It represents:

- Entities
- Attributes
- Relationships
- Constraints

It is usually converted into relational tables later.

Example:

Student applies to Company

Entities:

- Student

- Company

Relationship:

- Applies

Attributes:

- Student: student_id, name, cgpa
- Company: company_id, name, package
- Applies: application_date, status

13. Object-Oriented Data Model

Stores data as objects, similar to object-oriented programming.

An object contains:

- State: data/attributes
- Behavior: methods/functions

Features

- Supports classes and objects.
- Supports inheritance.
- Supports encapsulation.
- Useful for complex data.

Examples

- Multimedia databases
- CAD/CAM systems
- Scientific databases

Disadvantages

- Less commonly used than relational databases.
- Querying can be complex.
- Standards are less dominant than SQL.

14. Object-Relational Data Model

Combines relational model with object-oriented features.

Examples:

- PostgreSQL
- Oracle object-relational features

Supports:

- User-defined types

- Inheritance-like features
- Complex objects

15. Semi-Structured Data Model

Data does not follow a rigid table structure.

Examples:

- XML
- JSON

Example JSON

```
{
  "student_id": 1,
  "name": "Aman",
  "skills": ["SQL", "Java", "Python"]
}
```

Useful when records may have different attributes.

16. NoSQL Data Models

NoSQL databases are often used for large-scale, flexible, distributed data.

Types

1. Key-Value Store

Stores data as key-value pairs.

Example:

- Redis
- Amazon DynamoDB

```
student:1 -> {name: "Aman", cgpa: 8.5}
```

2. Document Store

Stores data as documents, usually JSON-like.

Example:

- MongoDB
- CouchDB

3. Column-Family Store

Stores data in columns grouped into column families.

Example:

- Cassandra
- HBase

4. Graph Database

Stores data as nodes and edges.

Example:

- Neo4j

Useful for:

- Social networks
- Recommendation systems
- Fraud detection

17. Constraints In DBMS

Constraints are rules applied to data to maintain correctness, validity, and integrity.

They prevent invalid data from entering the database.

Why Constraints Are Important

- Ensure data accuracy.
- Prevent invalid entries.
- Maintain relationships.
- Enforce business rules.
- Reduce application-side checks.

Types Of Constraints

1. Domain constraint
2. Key constraint
3. Entity integrity constraint
4. Referential integrity constraint
5. NOT NULL constraint
6. UNIQUE constraint
7. CHECK constraint
8. DEFAULT constraint
9. Assertion
10. Trigger-based constraint

18. Domain Constraint

A domain constraint restricts the values an attribute can take.

Example

```
cgpa DECIMAL(3,2) CHECK (cgpa >= 0 AND cgpa <= 10)
```

Valid values: 0.00 to 10.00

Invalid values: -1, 11

More Examples

- Age must be greater than 0.
- Gender must be from allowed values.
- Email must follow valid format.
- Salary must be positive.

19. Key Constraint

A key constraint ensures that tuples can be uniquely identified.

Super Key

A super key is any set of attributes that can uniquely identify a row.

Example:

In `Student(student_id, email, phone, name)`:

- {student_id}
- {email}
- {phone}
- {student_id, name}

All can be super keys if they uniquely identify students.

Candidate Key

A candidate key is a minimal super key.

Minimal means no unnecessary attribute is included.

Example:

- {student_id}
- {email}

If both uniquely identify students, both are candidate keys.

Primary Key

The candidate key chosen to uniquely identify rows.

Properties:

- Unique
- Not null
- Stable
- Minimal

Example:

```
student_id INT PRIMARY KEY
```

Alternate Key

Candidate keys not chosen as the primary key.

Example:

If `student_id` is primary key and `email` is also unique, then `email` is an alternate key.

Foreign Key

An attribute in one table that refers to the primary key of another table.

Example:

```
CREATE TABLE Application (  
  application_id INT PRIMARY KEY,  
  student_id INT,  
  company_id INT,  
  FOREIGN KEY (student_id) REFERENCES Student(student_id),  
  FOREIGN KEY (company_id) REFERENCES Company(company_id)  
);
```

Foreign keys maintain relationships between tables.

Composite Key

A key made of more than one attribute.

Example:

```
Enrollment(student_id, course_id)
```

Here `{student_id, course_id}` can be the primary key.

Surrogate Key

An artificial key added by the system.

Example:

`student_id, order_id, application_id`

Natural Key

A real-world attribute used as a key.

Example:

- Aadhaar number
- Email
- Roll number

For placements, use examples carefully: natural keys can change or may have privacy concerns.

20. Entity Integrity Constraint

Entity integrity says primary key values cannot be null.

Reason:

If primary key is null, the row cannot be uniquely identified.

Example:

```
student_id INT PRIMARY KEY
```

`student_id` cannot be null.

21. Referential Integrity Constraint

Referential integrity ensures that a foreign key value must either:

- Match an existing primary key value in the referenced table, or
- Be null, if allowed.

Example

If `Application.student_id` references `Student.student_id`, then an application cannot exist for a non-existing student.

Invalid Case

`Application` table:

application_id	student_id	company_id
1	999	10

If no student has `student_id = 999`, this violates referential integrity.

Actions On Delete/Update

CASCADE

If referenced row is deleted or updated, related rows are also deleted or updated.

```
ON DELETE CASCADE
```

SET NULL

Foreign key is set to null when referenced row is deleted.

```
ON DELETE SET NULL
```

RESTRICT / NO ACTION

Prevents deletion or update if related rows exist.

```
ON DELETE RESTRICT
```

SET DEFAULT

Foreign key is set to a default value.

```
ON DELETE SET DEFAULT
```

22. NOT NULL Constraint

Ensures a column cannot have null values.

Example:

```
name VARCHAR(100) NOT NULL
```

Use when an attribute is mandatory.

23. UNIQUE Constraint

Ensures all values in a column or group of columns are unique.

Example:

```
email VARCHAR(100) UNIQUE
```

Difference between primary key and unique:

Primary Key	Unique
Only one primary key per table	Multiple unique constraints allowed
Cannot be null	May allow null depending on DBMS
Identifies each row	Prevents duplicate values

24. CHECK Constraint

Ensures a condition is true for each row.

Example:

```
CHECK (cgpa BETWEEN 0 AND 10)
```

More examples:

```
CHECK (salary > 0)
CHECK (age >= 18)
CHECK (status IN ('Applied', 'Shortlisted', 'Rejected', 'Selected'))
```

25. DEFAULT Constraint

Provides a default value when no value is supplied.

Example:

```
status VARCHAR(20) DEFAULT 'Applied'
```

26. Assertions

An assertion is a database-level constraint that must always be true.

Example:

Total allocated interview slots should not exceed total available slots.

SQL standard supports assertions, but many commercial DBMSs do not fully implement them.

27. Triggers

A trigger is a procedure that automatically runs when a database event occurs.

Events:

- INSERT
- UPDATE
- DELETE

Example:

When a student's CGPA is updated, store the old value in an audit table.

Triggers can enforce complex constraints, but overuse can make systems hard to debug.

28. ER Model

ER stands for Entity-Relationship.

The ER model is used for conceptual database design.

It visually represents data using:

- Entities
- Attributes
- Relationships
- Constraints

ER diagrams help convert real-world requirements into database design.

29. Entity

An entity is a real-world object or concept that can be uniquely identified.

Examples:

- Student
- Company
- Course
- Department
- Employee
- Account

Entity Set

A collection of similar entities.

Example:

All students form the **Student** entity set.

Strong Entity

A strong entity has its own primary key.

Example:

Student(**student_id**, name, cgpa)

Weak Entity

A weak entity does not have a primary key of its own.

It depends on a strong entity for identification.

Example:

Dependent of an **Employee**.

A dependent may be identified by:

```
employee_id + dependent_name
```

Owner Entity

The strong entity on which a weak entity depends.

Example:

Employee is the owner entity of **Dependent**.

Identifying Relationship

The relationship between weak entity and owner entity.

Example:

Employee has **Dependent**

30. Attributes

Attributes describe properties of entities or relationships.

Example:

For **Student**:

- student_id
- name
- email
- cgpa

- department

Types Of Attributes

1. Simple Attribute

Cannot be divided further.

Example:

- age
- cgpa

2. Composite Attribute

Can be divided into smaller parts.

Example:

Name can be divided into:

- first_name
- middle_name
- last_name

Address can be divided into:

- street
- city
- state
- pincode

3. Single-Valued Attribute

Has only one value for each entity.

Example:

- date_of_birth
- roll_number

4. Multi-Valued Attribute

Can have multiple values for one entity.

Example:

- phone_numbers
- skills
- email_addresses

In ER diagrams, multi-valued attributes are often shown using double ovals.

In relational mapping, multi-valued attributes usually become a separate table.

Example:

```
StudentPhone(student_id, phone_number)
```

5. Derived Attribute

Can be calculated from other attributes.

Example:

Age can be derived from date of birth.

In ER diagrams, derived attributes are often shown using dashed ovals.

6. Stored Attribute

Stored directly in the database.

Example:

`date_of_birth` is stored, while `age` is derived.

7. Key Attribute

Attribute used to uniquely identify an entity.

Example:

`student_id`

31. Relationships

A relationship represents an association between entities.

Examples:

- Student enrolls in Course
- Employee works for Department
- Customer places Order
- Student applies to Company

Relationship Set

A collection of similar relationships.

Example:

All `applies_to` relationships between students and companies.

Degree Of Relationship

Number of entity sets participating in a relationship.

Unary / Recursive Relationship

Relationship involving one entity set.

Example:

Employee manages Employee.

Binary Relationship

Relationship involving two entity sets.

Example:

Student enrolls in Course.

Most common type.

Ternary Relationship

Relationship involving three entity sets.

Example:

Supplier supplies Part to Project.

32. Mapping Cardinality

Mapping cardinality describes how many entities of one set are associated with entities of another set.

1. One-To-One

One entity in A is related to at most one entity in B, and vice versa.

Example:

One person has one passport.

Person 1 ----- 1 Passport

2. One-To-Many

One entity in A can be related to many entities in B, but one entity in B is related to one entity in A.

Example:

One department has many students.

```
Department 1 ----- N Student
```

3. Many-To-One

Many entities in A are related to one entity in B.

Example:

Many students belong to one department.

```
Student N ----- 1 Department
```

4. Many-To-Many

Many entities in A can be related to many entities in B.

Example:

Many students apply to many companies.

```
Student M ----- N Company
```

In relational design, many-to-many relationships require a separate table.

Example:

```
Application(student_id, company_id, application_date, status)
```

33. Participation Constraints

Participation tells whether every entity must participate in a relationship.

Total Participation

Every entity in the entity set must participate in the relationship.

Example:

Every employee must belong to a department.

Shown by double line in ER diagrams.

Partial Participation

Some entities may not participate.

Example:

Some students may not apply to any company.

Shown by single line in ER diagrams.

34. ER Diagram Notations

Common Chen notation:

Concept	Symbol
Entity	Rectangle
Weak entity	Double rectangle
Relationship	Diamond
Identifying relationship	Double diamond
Attribute	Oval
Key attribute	Underlined oval
Multi-valued attribute	Double oval
Derived attribute	Dashed oval
Total participation	Double line
Partial participation	Single line

35. Converting ER Model To Relational Model

Rule 1: Strong Entity

Create a table for each strong entity.

Example:

```
Student(student_id, name, cgpa)
```

Rule 2: Weak Entity

Create a table for weak entity including:

- Its partial key
- Owner entity primary key as foreign key

Example:

```
Dependent(employee_id, dependent_name, age)
```

Primary key:

```
(employee_id, dependent_name)
```

Rule 3: One-To-One Relationship

Add primary key of one table as foreign key in the other table.

Prefer placing the foreign key on the side with total participation.

Example:

```
Person(person_id, name)
Passport(passport_id, person_id)
```

Rule 4: One-To-Many Relationship

Add primary key of the **one** side as foreign key in the **many** side.

Example:

```
Department(dept_id, dept_name)
Student(student_id, name, dept_id)
```

Rule 5: Many-To-Many Relationship

Create a new table for the relationship.

Example:

```
Student(student_id, name)
Company(company_id, company_name)
Application(student_id, company_id, application_date, status)
```

Primary key of **Application** can be:

```
(student_id, company_id)
```

Rule 6: Multi-Valued Attribute

Create a separate table.

Example:

```
Student(student_id, name)
StudentPhone(student_id, phone_number)
```

Primary key:

```
(student_id, phone_number)
```

Rule 7: Relationship With Attributes

Add relationship attributes to the relationship table.

Example:

For *Student applies to Company*, relationship attributes may be:

- application_date
- status
- interview_round

Table:

```
Application(student_id, company_id, application_date, status, interview_round)
```

36. ER Model Example For Placement System

Requirements

- A student belongs to one department.
- A department has many students.
- A company offers many jobs.
- A student can apply to many jobs.
- A job can receive applications from many students.
- Each application has status and application date.

Entities

Student

- student_id
- name

- email
- cgpa
- department_id

Department

- department_id
- department_name

Company

- company_id
- company_name
- location

Job

- job_id
- company_id
- role
- package
- minimum_cgpa

Application

- application_id
- student_id
- job_id
- application_date
- status

Relationships

- Department has Student: one-to-many
- Company offers Job: one-to-many
- Student applies to Job: many-to-many, resolved using **Application**

Relational Schema

```
CREATE TABLE Department (  
  department_id INT PRIMARY KEY,  
  department_name VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE Student (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  cgpa DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10),
```

```
    department_id INT NOT NULL,  
    FOREIGN KEY (department_id) REFERENCES Department(department_id)  
);  
  
CREATE TABLE Company (  
    company_id INT PRIMARY KEY,  
    company_name VARCHAR(100) NOT NULL,  
    location VARCHAR(100)  
);  
  
CREATE TABLE Job (  
    job_id INT PRIMARY KEY,  
    company_id INT NOT NULL,  
    role VARCHAR(100) NOT NULL,  
    package_lpa DECIMAL(6,2),  
    minimum_cgpa DECIMAL(3,2) CHECK (minimum_cgpa BETWEEN 0 AND 10),  
    FOREIGN KEY (company_id) REFERENCES Company(company_id)  
);  
  
CREATE TABLE Application (  
    application_id INT PRIMARY KEY,  
    student_id INT NOT NULL,  
    job_id INT NOT NULL,  
    application_date DATE NOT NULL,  
    status VARCHAR(20) DEFAULT 'Applied',  
    FOREIGN KEY (student_id) REFERENCES Student(student_id),  
    FOREIGN KEY (job_id) REFERENCES Job(job_id),  
    UNIQUE (student_id, job_id)  
);
```

37. Common Interview Questions

1. What is the difference between DBMS and file system?

DBMS provides structured storage, query processing, concurrency control, security, integrity constraints, backup, recovery, and reduced redundancy. File systems store data in files and require application programs to manually handle these features.

2. What is schema?

Schema is the structure or blueprint of a database. It defines tables, attributes, relationships, and constraints.

3. What is instance?

Instance is the actual data stored in the database at a particular point in time.

4. Difference between schema and instance?

Schema rarely changes and represents database design. Instance changes frequently and represents current data.

5. What is data independence?

Data independence is the ability to change schema at one level without affecting the next higher level.

6. Which is harder: physical or logical data independence?

Logical data independence is harder because changes in logical schema can affect application views and queries.

7. What is a data model?

A data model defines how data is structured, related, constrained, and manipulated.

8. Why is relational model popular?

It is simple, table-based, mathematically strong, supports SQL, and provides good data independence.

9. What is a constraint?

A constraint is a rule that restricts invalid data and maintains integrity.

10. What is primary key?

A primary key uniquely identifies each row in a table and cannot be null.

11. What is foreign key?

A foreign key is an attribute in one table that references the primary key of another table.

12. What is referential integrity?

Referential integrity ensures that foreign key values refer to existing primary key values or are null if allowed.

13. What is ER model?

The ER model is a conceptual design model that represents entities, attributes, relationships, and constraints.

14. What is a weak entity?

A weak entity cannot be uniquely identified by its own attributes and depends on a strong entity.

15. How do you convert many-to-many relationship into tables?

Create a separate relationship table containing primary keys of both participating entities as foreign keys.

38. Quick Revision Points

- DBMS manages databases.
- File systems are simple but weak for large, shared, secure, consistent data.
- Schema is structure; instance is current data.
- Schema is intension; instance is extension.
- Data models define how data is represented.
- Relational model stores data in tables.
- ER model is used for conceptual design.

- Constraints protect correctness.
- Primary key uniquely identifies rows.
- Foreign key connects tables.
- Entity integrity means primary key cannot be null.
- Referential integrity means foreign key must refer to valid data.
- One-to-many relationships place foreign key on many side.
- Many-to-many relationships need a new table.
- Multi-valued attributes need a separate table.
- Weak entities depend on strong entities.

39. One-Minute Summary

A DBMS is used to manage structured data efficiently and safely. Compared with file systems, it reduces redundancy, improves consistency, supports security, handles concurrent access, and provides backup and recovery. A schema is the database design, while an instance is the actual data at a given time. Data models define how data is organized; major models include hierarchical, network, relational, ER, object-oriented, semi-structured, and NoSQL models. Constraints enforce correctness through rules such as primary key, foreign key, unique, not null, check, and default. The ER model helps design databases using entities, attributes, relationships, cardinality, and participation constraints before converting the design into relational tables.